What's new

Two more apps: visualizing the zeta process and the motions of the heavens

Terence Tao · 16 July 2026

https://terrytao.wordpress.com/2026/07/16/two-more-apps-visualizing-the-zeta-process-and-the-motions-of-the-heavens/

tao2typ · converted to Typst 2026-10-14 (UTC)

I believe that the creation of visualization apps to illustrate mathematical or scientific concepts is a particularly favorable use case for modern coding agents, as many of the downside risks attached to other LLM use cases are limited:

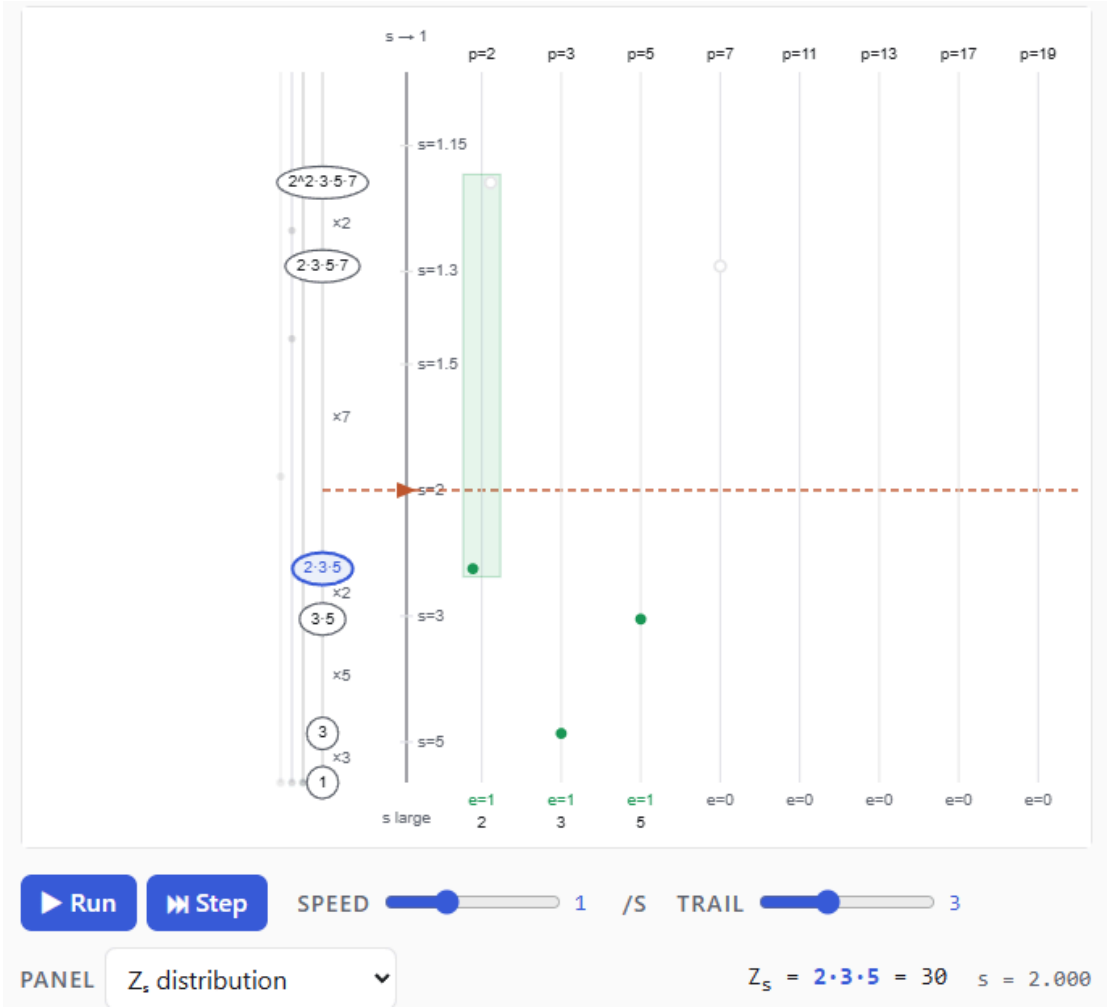
1. **Not mission-critical**. As such apps are not authorative sources of truth and only used for secondary purposes, a small positive error rate in the output can be acceptable.
2. **Stand-alone**. As the applets are not destined to be incorporated into a larger codebase or literature, the technical debt incurred by delegating all the coding to an LLM agent is bounded.
3. **End product is deterministic (and sandboxed)**. As the applets run on a deterministic language (Javascript), are sandboxed against file or internet access, and do not make any LLM calls at run-time, security and privacy concerns are minimal, and the applet can be maintained without continued premium LLM access or resource-intensive compute.
4. **Not replacing primary skills**. While deskilling is the tradeoff one accepts when relying on these tools to accelerate output, I am perfectly willing to forego the opportunity to keep my Javascript skills at a high level, as this is a tertiary skill for me at best in my chosen profession. (I continue to manually program in Lean and in Python to keep in practice with programming in general.)
5. **Not competing with humans**. To my knowledge, there is no existing human effort that is being duplicated by these applets (the activity in this direction appears to have peaked two decades ago).

I would however caution against unrestricted LLM use when one or more of the above five favorable situations is not in effect.

With these points in mind, I have used such an agent to create two further apps. The first app illustrates the "zeta process" that was introduced in my recent paper with Alexeev, Barreto, Li, Lichtman, Price, Shah, and Tang, though it was first discovered by an AI. For each $s > 1$, the zeta distribution Z_s is a random natural number with distribution

$$\mathbf{P}(Z_s = k) = \frac{1}{\zeta(s) k^s}.$$

It has long been known that this distribution has good number-theoretic properties: for instance, the number of times a given prime p divides Z_s has a geometric distribution of mean p^{-s}. However, the new observation is that these random variables Z_s can be chained together into a single stochastic process, which we call the "zeta process", which is an infinite divisibility chain. I used an agent to create an app to visualize this process:

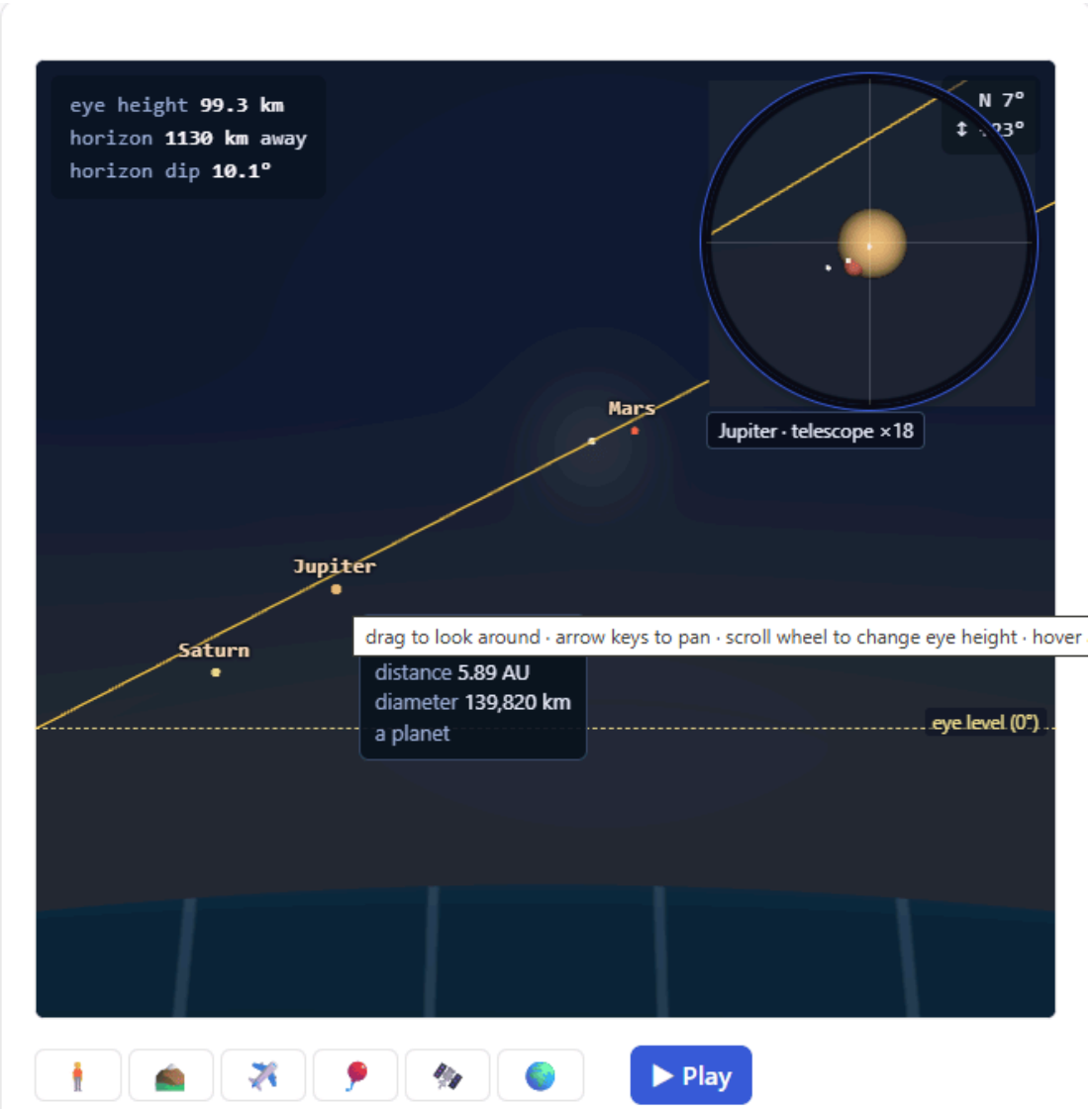


The underlying process is generated by several exponential random variables at each prime: in the above instantiation of the process, two such variables are visible at the prime $p = 2$, and one variable at the primes $p = 3, 5$. At a given choice of s, Z_s is formed by collecting all the variables below this threshold (and for which all predecessors also lie below the threshold); in the above illustration, this amounts to one variable at each of the primes $p = 2, 3, 5$, leading to $Z_2 = 30$ in this case. Additional visualizations in the app display the distribution of each Z_s, as well as the distribution of the hitting probability ν_Λ, which among other things can be used to give a quick solution to Erdős problem #1196.

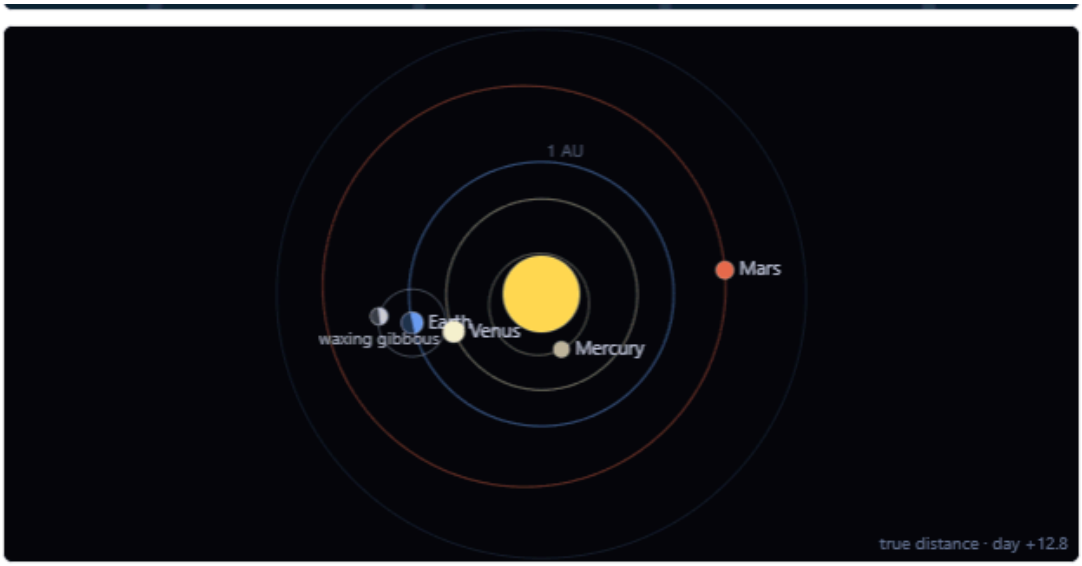
The second app is rather different in nature, and is a somewhat whimsical attempt to display the motion of the heavens, both at "human" scales of space and time, and at more "astronomical" scales (in which the motion of the planets in particular are more apparent). It is very loosely inspired by the game "Katamari Damacy", in which one absorbs both terrestrial and celestial objects of many different scales. Here is how the app typically looks at a human scale:



And here is how it looks when one's perspective leaves the Earth's atmosphere:



(As I did not want to render an entire explorable world in this app, the observer in the app is only limited to changing his or her size, from a human to a creature of comparable size to the Earth itself; they cannot move horizontally on the planet.) At the largest scales of space and time, the classic orrery diagram appears:



After lengthy conversations with the agent, I was able to implement many astronomical phenomena, including phases of the Moon, the effect of Earth's rotation against the fixed stars (though one can also stabilize one's view against those stars to see the Earth's rotation more directly), and so forth.

Comments (14)

Justin Bian · 16 July 2026

Great project :) !

Birke Heeren · 17 July 2026

Do you know the site of Dr. Schwebinghaus? There are a lot of interesting mathematical interactiv programs.
https://www.fraktalwelt.de/myhome/index.html

Anonymous · 17 July 2026

What percent of software do you think meets the first four favorable situations but not the fifth one (the "Not competing with humans" one)?
And if it's non-zero, do you think that would mean more software (that meets the first four situations) and same amount of software engineers? Or same amount of software (that meets the first four situations) and less software engineers? Or something else entirely?

Terence Tao · 18 July 2026

I am outsourcing this question to an AI.

Mason Porter · 18 July 2026

It's been close to a couple of decades since I saw someone reference Katamari Damacy. That game was an unexpected hit, and I never could particularly control the prince.
In terms of these various apps, there is also a sort of intermediary (which is much more common outside of theoretical mathematics, though still not terribly common) with animations that can be hosted e.g. in the Supplementary Materials of journals. So maybe mathematics will see a flux of AI animations placed in such settings. When SIAM started the journal SIAM Journal on Applications of Dynamical Systems (SIADS) a quarter of a century ago, they were hoping that people would take

advantage of its online-only publishing model by including animations with their articles, but it didn't materialize so much in practice.
A separate thing to point out in this vein (and this is something that could inspire more examples with modern AI tools) is a 'reimagining' by Bret Victor of the Watts–Strogatz 1998 'small-world' paper.
Here is his reimagining: https://worrydream.com/ScientificCommunicationAsSequentialArt/
Victor commented that the Watts–Strogatz paper had basically already reached what he viewed as a quality bound for a prose-only article (I don't agree with that statement, but that's a separate issue), and he added knobs for 'readers' to do things like interactively adjust parameter values (a rewiring probability, in this case). Further to apps, perhaps we can see more of this type of augmentation in the AI era. It would also be great for 'readers' to do this with papers in a semi-automated way even when authors don't.

 **J** · 19 July 2026

"(I continue to manually program in Lean and in Python to keep in practice with programming in general.)"
How do you decide which parts of your Lean or Python programming to outsource to AI, and how much of the work do you typically delegate in general?

 **Terence Tao** · 19 July 2026

The five criteria I listed in the blog post already provide a lot of guidance. For instance, if the Lean code I am writing is destined to be used in a larger code repository such as Mathlib, then it becomes more important to be able to understand the code itself and not just the final outputs, and heavy reliance of AI would usually not be a good idea. On the other hand, using AI tools to automatically review such code remains a reasonably low-risk activity, and I certainly take advantage of the ability of AI agents to review my code for adherence to stylistic guidelines etc..
My Youtube channel has some examples of various ways in which I use (or do not use) AI tools in my formalization work. (See also my "AI on the red team" advice in my paper with Klowden.)
For Python, my main concern is to avoid deskilling, as I do find it convenient to know how to do quick edits to Python code provided from other sources (including LLMs) to adapt them to my own needs, and to be able to visually inspect Python code for any serious red flags (e.g., unwanted I/O calls). But for the applets I have not found it necessary to directly edit or even review the Javascript provided, and rely on playtesting the final applet and interacting by text with the agent instead to bring the applet up to an acceptable level of quality for this non-mission-critical task.
(Incidentally, there are other desiderata to keep in mind beyond the five listed in the blog post, for instance whether prior art and intellectual property is being respected and properly attributed, but this is a relatively minor concern in these cases because the mathematical concepts being illustrated or coded originate from the human author, who has the expertise to provide suitable references.)

 **J** · 19 July 2026

I see roughly three ways of using AI in programming:

1. Writing the code entirely by hand, without AI assistance during implementation, perhaps using AI only afterward for debugging.
2. Writing the code oneself but using tools such as Copilot for assistance—for example, autocompleting tedious or routine parts.
3. Outsourcing most of the implementation to AI and mainly reviewing, testing, or correcting the resulting code afterward.

I'm curious whether your practice in Lean and Python tends to fall into one of these categories, or whether you use a different workflow.

Anonymous · 20 July 2026

These warnings might have been appropriate even 6 months ago, when coding agents were still less capable than humans. However, today there are many use cases that violate almost every item on that checklist, and even in those cases, using LLMs should be an absolute requirement. For example, any competent person working on mission-critical systems would be failing in their duty if they did not use LLMs to write and review code. In any codebase where security matters, LLMs are essential, because security has always been a huge liability at the mercy of human hands.
I think we have to recognize the big difference between what "vibecoding" entails and what software developers actually do when working with AI models. The work of software developers is not about one-off, only for personal use, disposable scripts with no tests or validation, generated in casual settings. If software developers tried to comply with the permitted use cases in this checklist, the only risk they would have to contend with is losing their job.
Even people who write code for a living have difficulty grasping the glaring deficiencies in human-generated code, so it is no wonder there is a generalized apprehension about letting an LLM do that job. That said, nobody who values their employment is letting the LLM run unrestricted, as this post implies.



Terence Tao · 21 July 2026

I would say that my own profession of mathematicians is significantly more than six months behind the software engineering profession right now with regards to median uptake of coding agents and adoption of current best practices. I do hope that over time the responsible use of AI in higher-risk situations becomes standardized, but at present there are enough early adopters of the technology in mathematics using them without sufficient awareness of downsides that caution is still advisable.

Anonymous · 22 July 2026

Absolutely. Actually, best practices seem to elude most professions, including a large number of software developers. Honestly, many people have to learn the dos and don'ts the hard way, not in high-stakes situations, of course, but by dealing with unexpected inconveniences. Also, the desire to adopt a new technology is not fully compatible with using it only in the least important scenarios. People need to figure out how to handle it, even in riskier situations. Eventually, this leads to embracing better habits (version control, backups, test-driven development, sandboxing, working with specs, setting file permissions, result verification, and so on).

Anonymous · 21 July 2026

Apply criteria 1-5 to the use of calculators. The same kind of arguments were made.
(not necessarily disagreeing with those criteria in principle, just that they will not be followed in general)

Guanyuming He · 30 July 2026

Hi Prof. Tao,
I'd like to add what I learned from software construction courses, that software not only needs to be safe today, but also safe in the unknown future. That especially relates to your 1 and 2 of the five principles.

One feature of modern software engineering is that a large portion of modern software systems are built with software reuse. A piece of software that is standalone and non-critical now may be used unexpectedly in the unknown future, especially when it becomes popular and people wants to modify/expand on it. Therefore, it is very important for software to come with some formal specification that encodes a rather precise understanding of what the software aims to achieve.
It is unclear to me how well LLM coding agents could generate such formal specifications for their output code, but so much is at least clear, that we will need to additionally verify their spec against their code, by which we will have to read their code anyway, a process that may outweight the benefit of having an agent coding for us in the first place.
Even if the original author who used the agent has no intention of the software being reused, whoever later decides to reuse it needs to conduct these steps, if that person wants the end product to be safe or correct.
Therefore, based on what I have learned, it seems that using agents creates a burden of still having to understand (verifying the software against a spec) the code, if the software is to be reused at all in the unknown future.
What do you think?
Guanyuming He



Terence Tao · 31 July 2026

My view is that now that we are in an era where code is very cheap to generate, many situations in which code re-use was previously warranted will be replaced by concept re-use instead; legacy code artefacts can still be mined for some structural insights, but it will likely be more efficient (and safer) to rewrite the code from scratch, using the preferred language, style conventions, programming practices, etc. of the modern project rather than import any technical debt from the legacy project. For instance, I have upstreamed some results to Mathlib that were first proven with some AI assistance, but while the AI proofs were useful for showing the general structure and pain points of the overall argument, it was still preferable to rewrite the code manually in many places, using the AI proofs merely as inspiration.
In each of my own apps coded via agents, I have also recorded a summmary of my conversation with that agent in which we discussed the specification and intent of the app, and in the cases where some usable output was generated (such as a JSON), I also directed the app to generate a detailed description of the output format (see the tab below the app in https://teorth.github.io/tao-web/apps/paper-diagram.html for an example). This seems sufficient for other humans or code agents to interface with the app (or build variants of it if necessary); having the original source code available as a reference can also be helpful but is not as crucial as it used to be, in my opinion.